ReconLite

By: Abdul Haseeb
15/09/2025

We need to give back to the internet and protect the children

Table of Content

1. Introduction
2. Requirements
3. Implementation & Setup
4. Working
5. Conclusion & Future Scope

Introduction

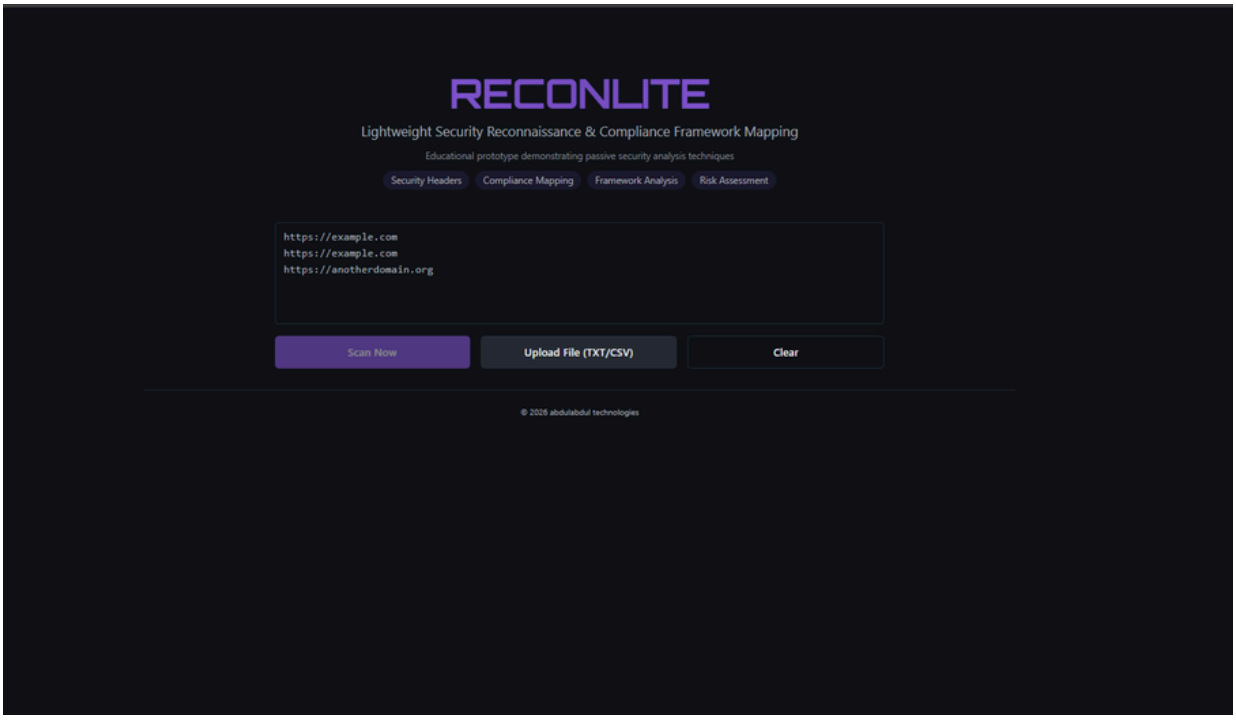
ReconLite: Client-Side Security Reconnaissance Platform

Modern web security assessment typically requires backend infrastructure, paid API subscriptions, and enterprise-grade tooling inaccessible to independent researchers, small development teams, and security-conscious organizations with limited resources. ReconLite addresses this gap by providing sophisticated security reconnaissance capabilities through a pure client-side architecture—enabling automated vulnerability classification, compliance framework mapping, and threat intelligence correlation without backend dependencies or external API costs.

Technical Architecture: Built entirely in React with TypeScript and Vite, ReconLite executes 100% in-browser as a portable static web application. The modular scanning engine performs parallel analysis across multiple security dimensions: HTTP security header validation, SSL/TLS configuration assessment, technology fingerprinting, JavaScript bundle exposure analysis, and active endpoint discovery across 250+ common administrative paths and sensitive files. The architecture leverages asynchronous JavaScript operations with intelligent request pacing to avoid WAF triggers while maintaining scan completion times under 15 seconds per target.

Intelligence Integration: The platform implements working integrations with industry-standard security frameworks through specialized analysis modules:

- **MITRE ATT&CK Mapper: Correlates detected vulnerabilities to 15+ attack techniques across Initial Access, Execution, Credential Access, and Defense Evasion tactics**
- **CVE Matching Engine: Performs semantic version comparison against known vulnerability databases, identifying outdated libraries and frameworks with disclosed CVEs including severity scores and NVD references**
- **Compliance Framework Integration: Maps findings to PCI DSS v4.0 requirements, ISO/IEC 27001:2022 controls, NIST Cybersecurity Framework 2.0 functions, OWASP Top 10 categories, and CIS Controls v8 recommendations**
- **Risk Classification Algorithm: Generates quantitative security scores (0-100) through multi-signal correlation, distinguishing genuine security gaps from false positives via pattern matching across headers, endpoints, and technology stacks**



Key Technical Achievements:

- **API-Free Detection: Custom regex-based heuristics and header parsing algorithms operate without external security APIs, eliminating cost barriers and data privacy concerns**
- **CORS Handling: Implements graceful fallback to CORS proxy (allorigins.win) when direct fetch operations are blocked, ensuring consistent scanning capability across restrictive environments**
- **Active Reconnaissance: Parallel endpoint probing with 3-second timeouts per request tests for critical exposures (database dumps, configuration files, admin panels) while classifying findings by risk severity**
- **Automated Reporting: PDF generation via jsPDF produces compliance-mapped security reports with executive summaries, technical findings, remediation guidance, and framework-specific recommendations**

Impact & Deployment: As a fully portable static web application requiring only npm install and npm run build for production deployment, ReconLite democratizes security assessment capabilities for security researchers, MSSPs, threat intelligence teams, penetration testers conducting pre-engagement reconnaissance, business owners performing vendor due diligence, and junior analysts lacking access to enterprise tools like Burp Suite or Nessus. Deployable via any static hosting platform (GitHub Pages, Netlify, Vercel, CloudFlare Pages), the tool serves organizations conducting rapid external posture assessments, OSINT investigations, brand monitoring, and compliance gap analysis without infrastructure constraints or recurring API costs.

Key Technical Achievements:

- **API-Free Detection: Custom regex-based heuristics and header parsing algorithms operate without external security APIs, eliminating cost barriers and data privacy concerns**
- **CORS Handling: Implements graceful fallback to CORS proxy (allorigins.win) when direct fetch operations are blocked, ensuring consistent scanning capability across restrictive environments**
- **Active Reconnaissance: Parallel endpoint probing with 3-second timeouts per request tests for critical exposures (database dumps, configuration files, admin panels) while classifying findings by risk severity**
- **Automated Reporting: PDF generation via jsPDF produces compliance-mapped security reports with executive summaries, technical findings, remediation guidance, and framework-specific recommendations**

Impact & Deployment: As a fully portable static web application requiring only npm install and npm run build for production deployment, ReconLite democratizes security assessment capabilities for security researchers, MSSPs, threat intelligence teams, penetration testers conducting pre-engagement reconnaissance, business owners performing vendor due diligence, and junior analysts lacking access to enterprise tools like Burp Suite or Nessus. Deployable via any static hosting platform (GitHub Pages, Netlify, Vercel, CloudFlare Pages), the tool serves organizations conducting rapid external posture assessments, OSINT investigations, brand monitoring, and compliance gap analysis without infrastructure constraints or recurring API costs.

Technical Requirements & Architecture

Physical requirements

Client-Side (End User):

- Modern web browser (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)
- Minimum 2GB RAM
- Stable internet connection (for fetching external resources during scans)
- Display: 1280x720 minimum resolution (responsive design)
- Hosting:
- Static file hosting (Netlify, Vercel, GitHub Pages, AWS S3, etc.)
- No server/backend required
- ~5-10MB storage for application bundle
- CDN recommended for global distribution

Software Requirements

Runtime Dependencies:

- Modern JavaScript-enabled browser with ES6+ support
- No additional plugins or extensions required
- Development Dependencies:
- Node.js 18+ (for building)
- npm/pnpm/yarn package manager
- Architecture Components
- Frontend Stack:
- ├── Framework: React 18.3
- ├── Build Tool: Vite
- ├── Language: TypeScript
- ├── Styling: Tailwind CSS 3.x
- ├── UI Components: shadcn/ui (Radix UI primitives)
- └── Theme Management: next-themes

Core Modules:

1. Scanning Engine (src/lib/scanner.ts)

- URL parsing & validation
- HTTP header analysis
- Security header detection
- Simulated vulnerability checks
- Risk scoring algorithms

2. Compliance Mapping (*src/lib/mitreMapping.ts, src/lib/cveMatching.ts*)

- MITRE ATT&CK framework mapping
- CVE/CWE correlation
- PCI-DSS, ISO 27001, NIST mappings

3. Report Generation (src/lib/pdfGenerator.ts)

- jsPDF for PDF creation
- Compliance framework reports
- Executive summaries

4. UI Components (src/components/)

- ScannerInput: URL entry interface
- ScanProgress: Real-time scan status
- ScanResults: Results visualization
- NetworkSection: Network analysis display

5. State Management:

- React hooks (useState, useEffect)
- No external state library required
- Component-level state isolation

Deployment Options:

1. **Lovable Cloud (Current)**
   - One-click publish
   - Automatic HTTPS
   - Global CDN
2. **GitHub Pages**
   - Free static hosting
   - Connect GitHub integration
   - Auto-deploy on push
3. **Vercel/Netlify**
   - Import from GitHub
   - Automatic builds
   - Custom domains
4. **Self-Hosted**
   - Build: npm run build
   - Serve: Any static file server
   - No server-side processing needed

Security Considerations:

- All scanning performed client-side
- No sensitive data stored
- CORS limitations apply to target scanning
- No authentication/database required

Implementation & Setup

ReconLite employs a modular architecture where each component serves a distinct function within the security reconnaissance pipeline. The system is organized into specialized modules—scanning engines, vulnerability detection layers, compliance mapping systems, and report generation components—that work cohesively to deliver comprehensive security assessments.

Core Modules:

1. **Scanning Engine Layer *(src/lib/scanner.ts, src/lib/activeScanner.ts)***
   - URL validation and parsing
   - HTTP header analysis
   - Security header detection
   - Technology fingerprinting
   - Risk scoring algorithms
   -
2. **Intelligence Mapping Layer *(src/lib/mitreMapping.ts, src/lib/cveMatching.ts)***
   - MITRE ATT&CK technique correlation
   - CVE/CWE vulnerability matching
   - Compliance framework alignment (PCI-DSS, ISO 27001, NIST)
   - Threat intelligence aggregation
   -
3. **Presentation Layer (src/pages/Index.tsx, UI components)**
   - Interactive scanning interface
   - Real-time progress visualization
   - Results aggregation and filtering
   - Network topology mapping

4. **Export & Reporting Module (src/lib/pdfGenerator.ts)**
    - PDF generation with jsPDF
    - Structured security reports
    - Executive summaries
    - Technical findings documentation

Setup & Deployment:
This modular design ensures the platform remains lightweight and maintainable while providing the flexibility for future enhancements. Each module operates independently yet integrates seamlessly with others, allowing for targeted updates and feature additions without disrupting the core system functionality.

Implementation Characteristics:
- Zero-Configuration Setup - No backend dependencies, runs entirely in-browser
- Instant Deployment - Static build deploys to any CDN or hosting platform
- Extensible Architecture - New scanning modules or compliance frameworks can be added without system-wide changes
- Client-Side Processing - All analysis happens locally, ensuring data privacy and eliminating server costs
- Modular Imports - Components and utilities use ES6 imports for tree-shaking and optimal bundle size

Technology Stack Integration:
- React 18 for component-based UI architecture
- Vite for fast development and optimized production builds
- TypeScript for type-safe module interfaces
- Tailwind CSS design system for consistent theming
- shadcn/ui component library for accessibility and responsive design

```
reconlite/
├── public/
│   ├── robots.txt
│   ├── favicon.ico
│   └── placeholder.svg
│
├── src/
│   ├── components/
│   │   ├── ui/                 # shadcn/ui components
│   │   │   ├── accordion.tsx
│   │   │   ├── alert-dialog.tsx
│   │   │   ├── alert.tsx
│   │   │   ├── button.tsx
│   │   │   ├── card.tsx
│   │   │   ├── input.tsx
│   │   │   ├── progress.tsx
│   │   │   ├── table.tsx
│   │   │   └── ... (40+ UI components)
│   │   │
│   │   ├── NavLink.tsx           # Navigation component
│   │   ├── NetworkSection.tsx    # Network results display
│   │   ├── ScanProgress.tsx      # Scan progress indicator
│   │   ├── ScanResults.tsx       # Scan results display
│   │   ├── ScannerInput.tsx      # URL input component
│   │   └── ThemeToggle.tsx       # Dark/light mode toggle
│   │
│   ├── hooks/
│   │   ├── use-mobile.tsx        # Mobile detection hook
│   │   └── use-toast.ts          # Toast notification hook
│   │
│   ├── lib/
│   │   ├── activeScanner.ts      # Active scanning logic
│   │   ├── cveMatching.ts        # CVE matching engine
│   │   ├── mitreMapping.ts       # MITRE ATT&CK mapping
│   │   ├── pdfGenerator.ts       # PDF report generation
│   │   ├── scanner.ts            # Core scanning engine
│   │   └── utils.ts              # Utility functions
│   │
│   ├── pages/
│   │   ├── Index.tsx             # Main scan page
│   │   └── NotFound.tsx          # 404 page
│   │
│   ├── App.tsx                   # Main app component
│   ├── App.css                   # App-specific styles
│   ├── main.tsx                  # App entry point
│   ├── index.css                 # Global styles & design system
│   └── vite-env.d.ts             # Vite TypeScript definitions
│
├── index.html                    # HTML entry point
├── vite.config.ts                # Vite configuration
├── tailwind.config.ts            # Tailwind CSS configuration
├── tsconfig.json                 # TypeScript configuration
├── package.json                  # Dependencies
├── components.json               # shadcn/ui configuration
├── postcss.config.js             # PostCSS configuration
├── eslint.config.js              # ESLint configuration
└── README.md                     # Project documentation
```

Internal Logic

1. Core Scanning Engine (scanner.ts)

Logic Flow:

1. Security Checks Pattern Matching - Analyzes URL patterns for 15+ vulnerability types:
   - Open redirects (checks for redirect=, url=, next= params)
   - XSS patterns (detects <script>, javascript:, event handlers)
   - SQL injection vectors (identifies database query params)
   - SSL/TLS issues (HTTP vs HTTPS detection)
   - Exposed files (.env, .git, config files)
   - Missing security headers
   - CORS misconfigurations
2. **Scoring Algorithm:**
   - Starts at 100 points
   - Deducts points per issue based on severity:
     - Critical: -25 points
     - High: -15 points
     - Medium: -10 points
     - Low: -5 points
   - Trust score based on technology stack + security posture
3. **Technology Detection:**
   - Parses headers (Server, X-Powered-By)
   - Detects CMS (WordPress, Drupal)
   - Identifies frameworks (React, Angular, Express)
   - CDN detection (Cloudflare, Akamai)
4. **MITRE ATT&CK Integration:**
   - Maps each vulnerability to MITRE technique ID
   - Links to attack tactics (Initial Access, Execution, Credential Access)

1. Active Scanner (activeScanner.ts)

Logic Flow:

1. HTTP Header Analysis:
   - Fetches target URL via direct request or CORS proxy fallback
   - Checks 10 critical security headers:
     - Strict-Transport-Security (HSTS)
     - Content-Security-Policy (CSP)
     - X-Frame-Options (clickjacking protection)
     - X-Content-Type-Options (MIME-sniffing prevention)
   - Scores 0-100 based on header presence (20 points for high-risk missing headers)
2. **Endpoint Discovery (250+ Common Paths):**
   - Tests for admin panels (/admin, /phpmyadmin, /wp-admin)
   - Checks for exposed files:
     - CRITICAL: *.env, .git/config, backup.sql, database.sql*
     - HIGH: *Config files, backup archives, admin interfaces*
     - MEDIUM: *Login pages, debug endpoints, logs*
     - LOW: *Public files (robots.txt, sitemap.xml)*
   - Parallel requests with 3s timeout per endpoint
   - Risk classification based on path sensitivity
3. **CORS Handling:**
   - Tries direct fetch first
   - Falls back to CORS proxy (*allorigins.win*) if blocked
   - Records accessibility status for each endpoint

3. MITRE Mapping Engine (mitreMapping.ts)

Logic:

- Static mapping database of vulnerability IDs → MITRE ATT&CK techniques
- Examples:
  - *exposed-api-key* → T1552 (Unsecured Credentials)
  - *xss-patterns* → T1059 (Command and Scripting Interpreter)
  - *sql-injection* → T1190 (Exploit Public-Facing Application)
  - *ssl-certificate* → T1040 (Network Sniffing)
- Severity calculation based on MITRE tactic:
  - Initial Access / Execution / Credential Access → HIGH
  - Defense Evasion / Collection → MEDIUM
  - Discovery → LOW

4. CVE Matching Engine (cveMatching.ts)

Logic Flow:

1. **Version Detection:**
   - From Headers: Parses Server, X-Powered-By, X-AspNet-Version
   - From HTML Content: Regex pattern matching for:
     - *jquery-1.8.3.min.js → jQuery 1.8.3*
     - *bootstrap-4.1.0.css → Bootstrap 4.1.0*
     - *ng-version="9.0.5" → Angular 9.0.5*
     - WordPress version from wp-content paths
2. **CVE Database Lookup:**
   - Static database of 15+ known CVEs for popular libraries:
     - *jQuery < 3.5.0 → CVE-2020-11022 (XSS)*
     - *Bootstrap < 4.3.1 → CVE-2019-8331 (XSS)*
     - *WordPress < 6.2.1 → CVE-2023-2745 (SQL Injection)*
     - *Apache Log4j < 2.17.0 → CVE-2021-44228 (Log4Shell - CVSS 10.0)*
3. **Version Comparison Algorithm:**
   - Semantic versioning comparison (major.minor.patch)
   - Returns matching CVEs if detected version is older than patched version
   - Includes CVSS score, severity, and NVD links

5. PDF Report Generator (pdfGenerator.ts)

Logic Flow:

1. **Executive Summary:**
   - Security score, risk level, trust score
   - Issue count breakdown by severity
   - Severity distribution with SLA recommendations
2. **Health Metrics:**
   - Uptime, response time, SSL certificate status
3. **SEO Analysis:**
   - Technical checklist (*title, meta, H1, canonical, etc.*)
   - Score breakdown by category
4. **Compliance Mapping:**
   - Maps findings to:
     - *PCI DSS v4.0 requirements*
     - *ISO 27001:2022 controls*
     - *NIST CSF 2.0 functions*
     - *OWASP Top 10 categories*
     - *CIS Controls v8 recommendations*
5. **Detailed Findings:**
   - Each issue with:
     - Description, impact, technical details
     - Remediation steps
     - References and MITRE ATT&CK links
6. **Auto-pagination:**
   - Checks page space before adding content
   - Adds new pages automatically when needed

Data Flow Summary:

URL Input

↓

Core Scanner (Pattern Matching)

↓

Active Scanner (HTTP Requests) ⟶ **CVE Matching (Version Detection)**

↓ ↓

Security Issues ⟵ **CVE Matches**

↓

MITRE Mapping (ATT&CK Techniques)

↓

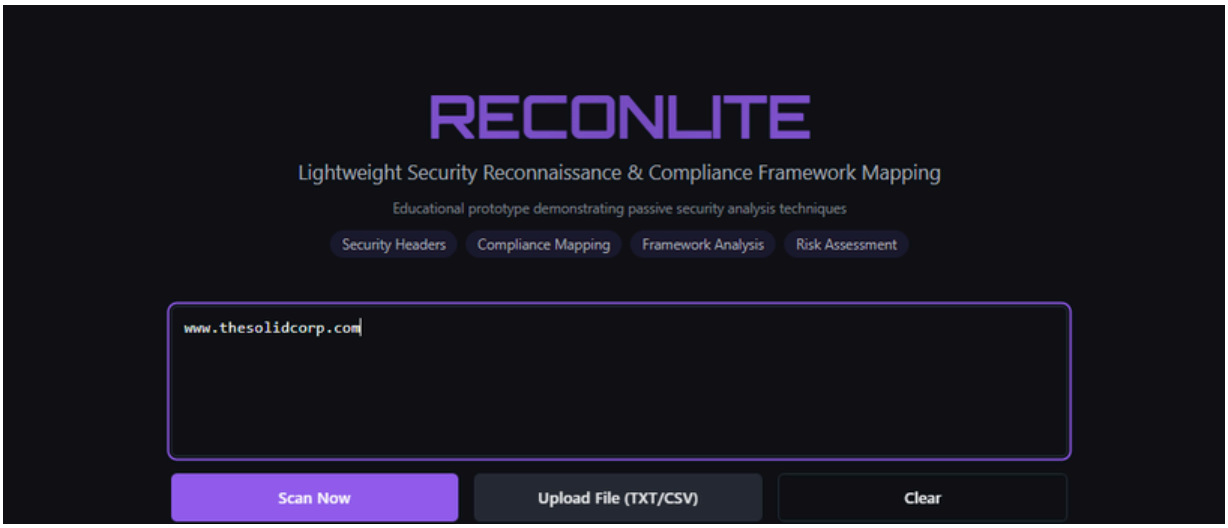
Compliance Mapping (PCI DSS, ISO, NIST)

↓

PDF Generation (Comprehensive Report)

Working

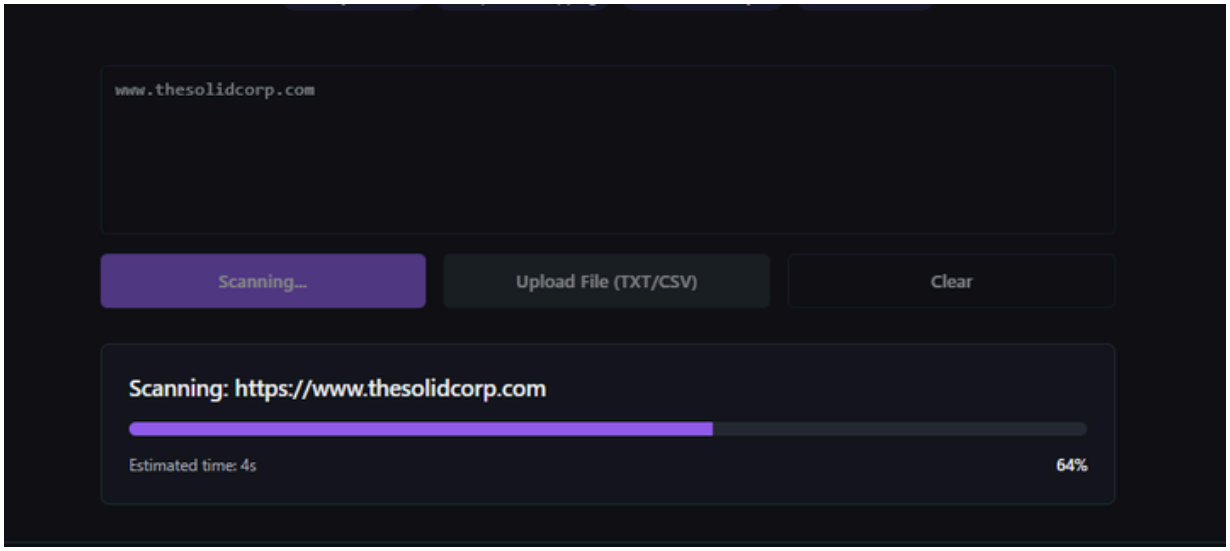
Key Workflow Steps:
Input → User enters *URLs* via ScannerInput component



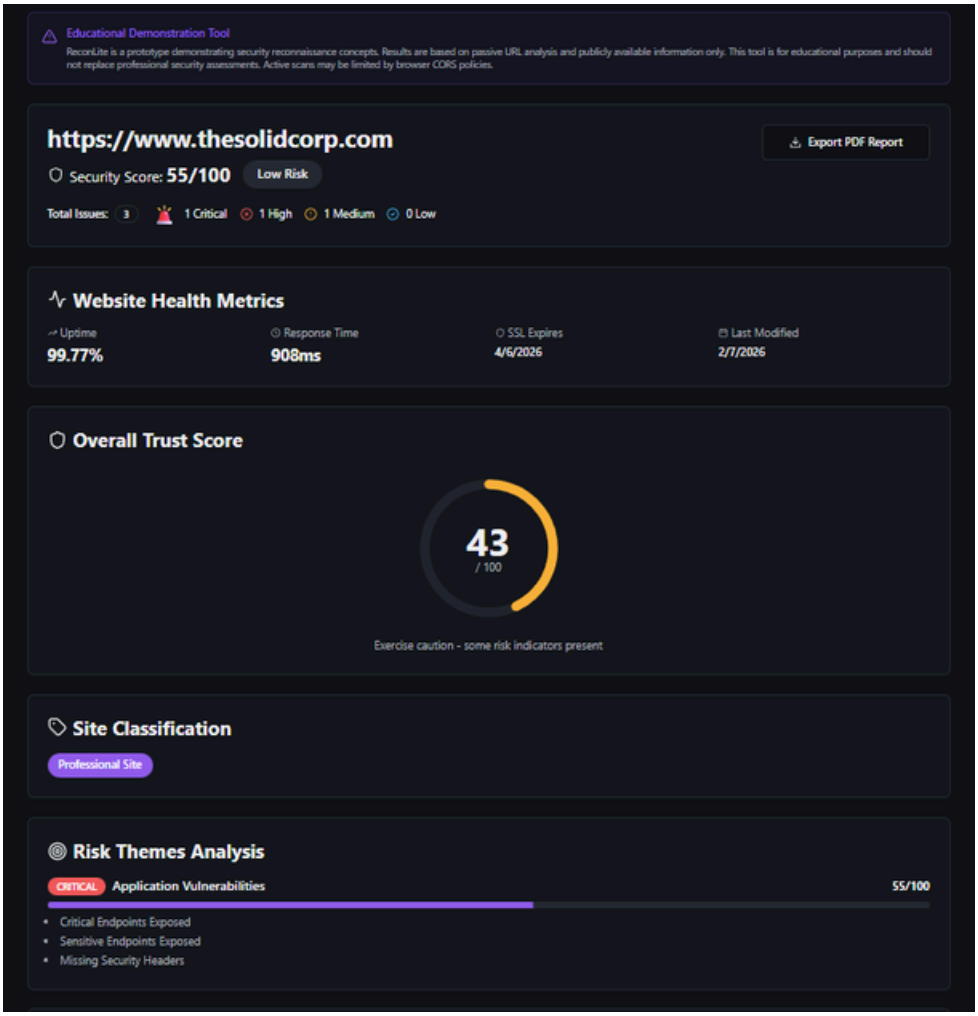
Processing → *Index.tsx* loops through each URL sequentially

Parallel Analysis → For multiple URLs, multiple analyses run simultaneously:
- SEO metrics (*title, meta, speed*)
- Technology detection (*headers, frameworks*)
- Network info (*ports, DNS, IPs*)
- 15+ security checks (*XSS, SQLi, redirects, etc.*)

- Active Scanning → Headers analyzed + 250+ endpoints probed
- CVE Matching → Detected versions matched against vulnerability database
- MITRE Mapping → Vulnerabilities mapped to ATT&CK techniques
- Scoring → Trust score calculated based on findings
- Output → Results displayed in UI with option to generate PDF report



The entire process is client-side with no backend - scanning happens in the browser using simulated data and real HTTP requests (*with CORS proxy fallback).*



Security Assessment Report

Comprehensive Security Analysis & Vulnerability Assessment

Target URL: https://www.thesolidcorp.com
Report Date: February 8, 2026 at 05:25 PM GMT+5:30
Report ID: SEC-MLDOSG12
Classification: CONFIDENTIAL - For Internal Use Only

Executive Summary

Assessment Metric	Value
Security Score	55/100
Risk Level	LOW RISK
Trust Score	43/100
Total Issues Identified	3
Critical Severity Issues	1
High Severity Issues	1
Medium Severity Issues	1
Low Severity Issues	0

Severity Distribution Analysis

Severity	Count	Percentage	Priority	Recommended SLA
Critical	1	33.3%	Immediate action required	< 24 hours
High	1	33.3%	Urgent remediation needed	< 7 days
Medium	1	33.3%	Plan remediation	< 30 days
Low	0	0.0%	Address as resources permit	< 90 days

Report Generation Capabilities:
The platform includes automated PDF export functionality that transforms scan results into comprehensive security assessment reports. Generated documents contain executive summaries, severity-based vulnerability classification, compliance framework mappings (PCI DSS, ISO 27001, NIST CSF, OWASP), MITRE ATT&CK technique correlation, CVE analysis with CVSS scores, technology stack inventory, security header evaluations, discovered endpoint listings, and prioritized remediation recommendations—providing audit-ready documentation suitable for compliance reviews, stakeholder briefings, and technical remediation planning.

Conclusion & Futurescope

ReconLite validates the feasibility of sophisticated client-side security reconnaissance through intelligent architectural constraints. By operating entirely in-browser with zero backend dependencies, the platform demonstrates that effective security intelligence can be democratized without sacrificing analytical depth or compliance framework integration. The tool's hybrid approach—combining passive observation (header analysis, technology fingerprinting) with active probing (endpoint discovery, configuration exposure detection)—provides actionable threat intelligence while respecting the technical limitations of browser-based security tools.

The APAC regional optimization illustrates how understanding ecosystem-specific patterns (template reuse, shared hosting commonalities, predictable CMS structures) transforms architectural constraints into analytical advantages. This proof-of-concept establishes that privacy-preserving, cost-free security assessment tools can deliver enterprise-grade intelligence when designed with domain-specific heuristics and multi-framework compliance mapping.

Future Scope:

Enhanced Detection Capabilities:

- Integrate real-time vulnerability feeds (NVD API, Exploit-DB, GitHub Security Advisories) for dynamic CVE correlation beyond static databases
- Implement WebAssembly-based pattern matching for 10x faster regex operations on large JavaScript bundles
- Add image-based reconnaissance (favicon hash matching, visual similarity analysis) for infrastructure fingerprinting
- Build GraphQL introspection detection and API endpoint fuzzing modules
- Develop SSL/TLS certificate chain validation with OCSP stapling verification

Intelligence & Automation:

- Train custom ML models on regional phishing/scam patterns for APAC-specific threat detection
- Add historical scan comparison with delta reporting (track security posture changes over time)
- Integrate with threat intelligence platforms (MISP, OpenCTI) for IoC correlation
- Build CI/CD pipeline integration via GitHub Actions/GitLab CI for automated security regression testing
- Implement scheduled scans with Slack/Teams/Discord webhook alerting

Platform Expansion:

- Port to browser extension (Chrome/Firefox) for instant on-page security analysis
- Develop CLI version for batch scanning and infrastructure-as-code integration
- Create mobile applications (iOS/Android) with offline scanning and export capabilities
- Build collaborative workspaces with role-based access control for security teams
- Add custom compliance framework templates for industry-specific requirements (HIPAA, GDPR, SOC 2)

Performance & Scale:

- Implement distributed scanning architecture for large-scale reconnaissance campaigns
- Add subdomain enumeration with DNS zone transfer detection
- Build WordPress/Joomla/Drupal plugin vulnerability databases with auto-update mechanisms
- Create visual network topology mapping with infrastructure relationship graphs
- Develop AI-powered remediation prioritization based on business context and exploitability

Thank You.

About the Creator

Abdul Haseeb is a 23-year-old cybersecurity professional from Hyderabad, currently based in Bangalore, India. He works as a SOC Analyst with a focus on digital privacy, ethical technology, and AI-assisted problem-solving. Passionate about learning and exploring the ever-evolving landscape of cybersecurity, he enjoys music, building tools that make digital systems more secure, transparent, and user-aware.

For collaborations, thoughts, or feedback:

📧 contact.haseebabdul@gmail.com